

Build Serverless REST API Microservices with Amazon API Gateway and AWS Lambda

Summary:-

In this lab, you will build a serverless Product API using Amazon API Gateway REST API and AWS Lambda. You will create a Lambda function, configure REST API resources and HTTP methods, integrate the API with Lambda using Lambda Proxy Integration, explore the API Gateway management console, deploy the API, inspect incoming REST API event objects using CloudWatch Logs, update the Lambda function to process different HTTP methods, enable request validation for query string parameters, and test the API using both the API Gateway console and Postman.

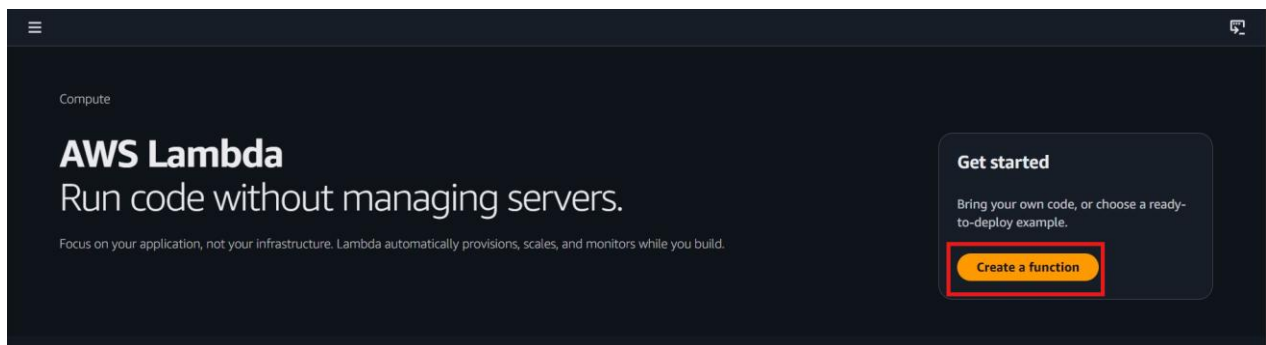
Let's Start:-

- Create a Lambda function for the Product API.
- Create an Amazon API Gateway REST API.
- Configure Product resources and HTTP methods.
- Integrate API Gateway methods with the Lambda function.
- Deploy the REST API.
- Test the REST API using the browser and Postman.
- Capture incoming REST API event objects in CloudWatch Logs.
- Update the Lambda function to handle GET, POST, and DELETE requests.
- Enable request validation for query string parameters.
- Test request validation and API responses.

Lab Steps:-

Step 1 – Create the Lambda Function

- Open the AWS Lambda console.
- Click Create function.



- Select Author from scratch.
- Enter the function name:

Product_function

- Select Node.js 24.x as the runtime.

Create function [Info](#)

Choose one of the following options to create your function.

- ☒ **Author from scratch**
Start with a simple Hello World example.
- ☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.
- ☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom** execution role under **Additional settings**.

- Leave the remaining settings as default.
- Click Create function.
- Under the Code tab, add the following line inside the Lambda handler to log incoming REST API events.
`console.log(JSON.stringify(event));`

Code source [Info](#) [Open in Visual Studio Code](#) [Update](#)

EXPLORER

- `JS index.mjs`

DEPLOY [Undeployed](#)

[Deploy \(Ctrl+Shift+U\)](#)

[Test \(Ctrl+Shift+I\)](#)

TEST EVENTS [NONE SELECTED]

```
1 export const handler = async (event) => {  
2   console.log(JSON.stringify(event));  
3   const response = {  
4     statusCode: 200,  
5     body: JSON.stringify('Hello from Lambda!'),  
6   };  
7   return response;  
8 };  
9
```

- Click Deploy.

Step 2 – Create the REST API

- Open the API Gateway console.
- Click Create API.

API Gateway > APIs > Create API > Create REST API

API details

☒ **New API**
Create a new REST API.

☐ **Clone existing API**
Create a copy of an API in this AWS account.

☐ **Import API**
Import an API from an OpenAPI definition.

☐ **Example API**
Learn about API Gateway with an example API.

API name
Product-API

Description - optional

API endpoint type
Regional APIs are deployed in the current AWS Region. Edge-optimized APIs route requests to the nearest CloudFront Point of Presence. Private APIs are only accessible from VPCs.
Regional

- Click **Create API**.

Step 3 – Create the Product Resource

- Select the root resource (/).
- Click **Actions** → **Create Resource**.

API Gateway > APIs > Resources - Product-API (dvpc9dgwf5)

Successfully created REST API 'Product-API (dvpc9dgwf5)'

Resources

Create resource

Resource details

Path: /

Resource ID: mabryzcrpl

- Configure:

Resource name: Product

API Gateway > APIs > Resources - Product-API (dvpc9dgwf5) > Create resource

Create resource

Resource details

☒ **Proxy resource** Info
Proxy resources handle requests to all sub-resources. To create a proxy resource use a path parameter that ends with a plus sign, for example {proxy+}.

Resource path: /

Resource name: Product

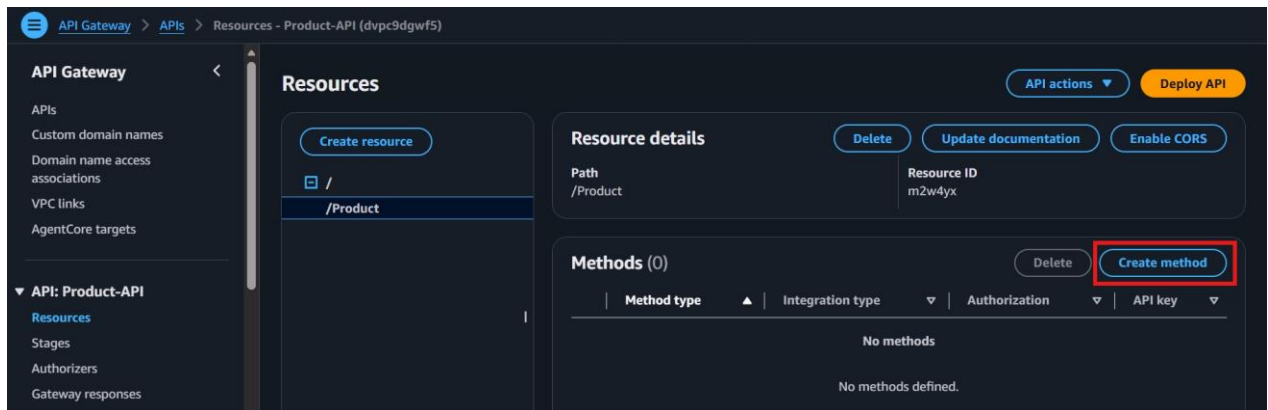
☐ **CORS (Cross Origin Resource Sharing)** Info
Create an OPTIONS method that allows all origins, all methods, and several common headers.

Cancel Create resource

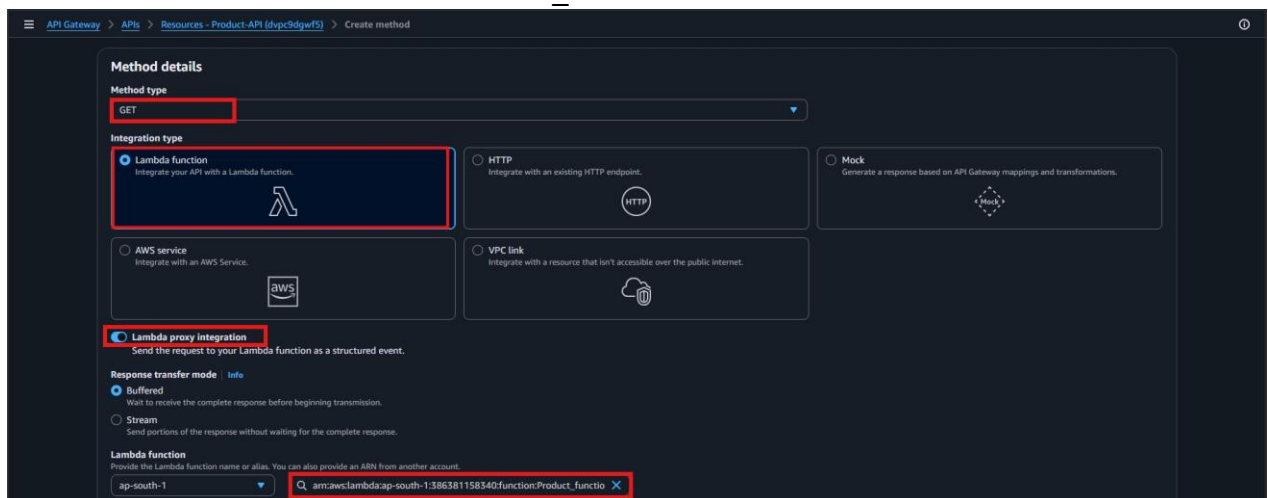
- Click **Create Resource**.

Step 4 – Create GET and POST Methods

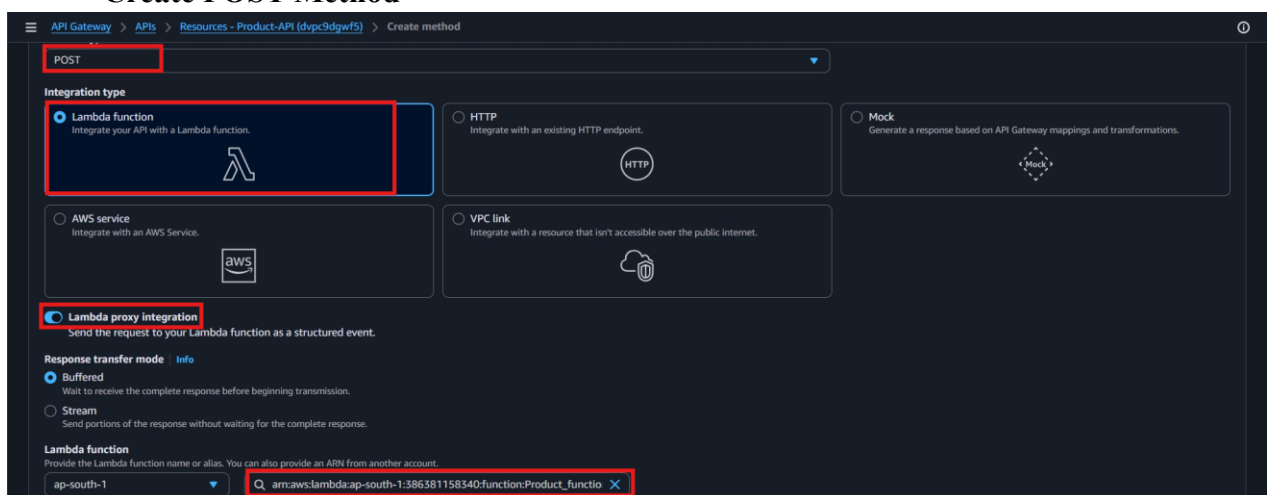
- Under the /Product resource:
 - Create GET Method
- Click **Actions** → **Create Method**.



- Select GET.
- Configure:
 - Integration Type: **Lambda Function**
 - Enable **Lambda Proxy Integration**
 - Lambda Function: **Product_Function**



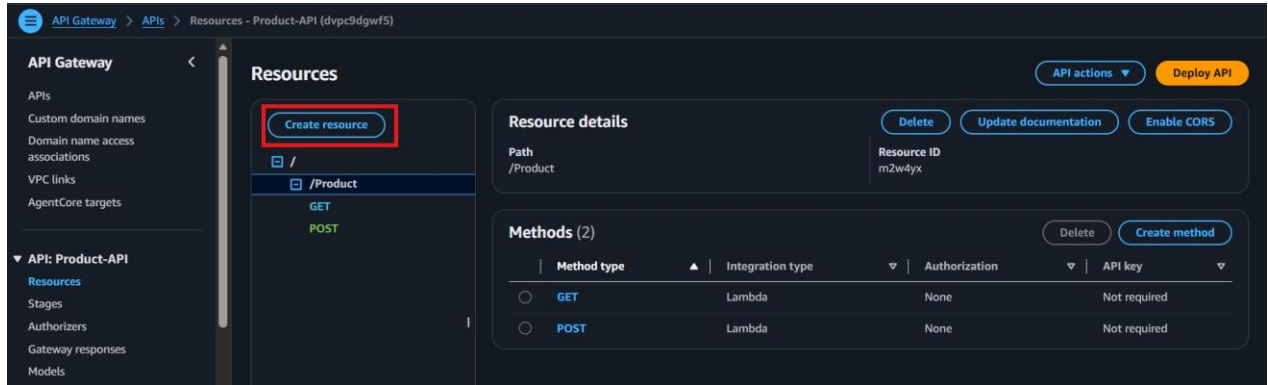
- Click Create method.
- Create POST Method



- Repeat the same steps and create a **POST** method using the same Lambda function. Verify that both methods are integrated with the Lambda function.

Step 5 – Create the Product ID Resource

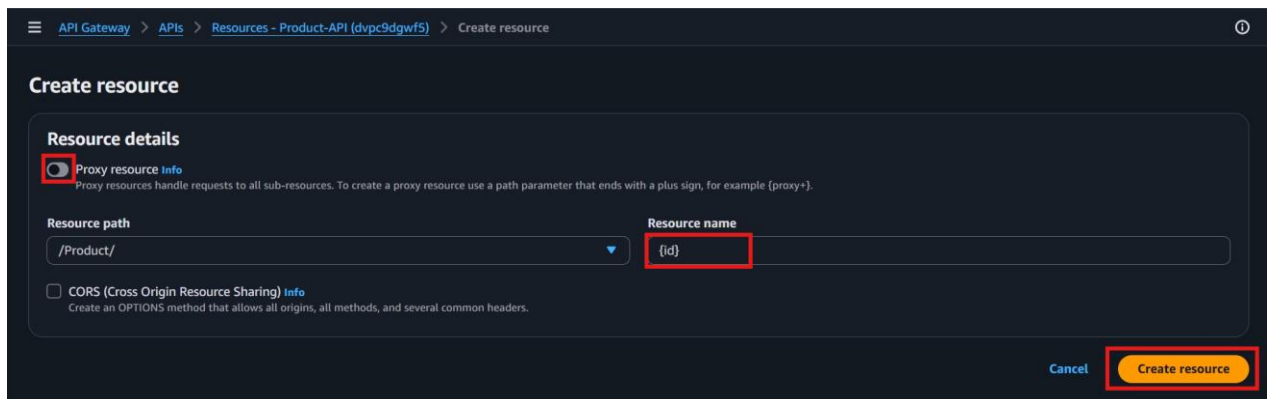
- Select the **/product** resource.



- Click **Actions** → **Create Resource**.
- Enter:

Setting	Value
Resource Name	{id}
Resource Path	/product/{id}

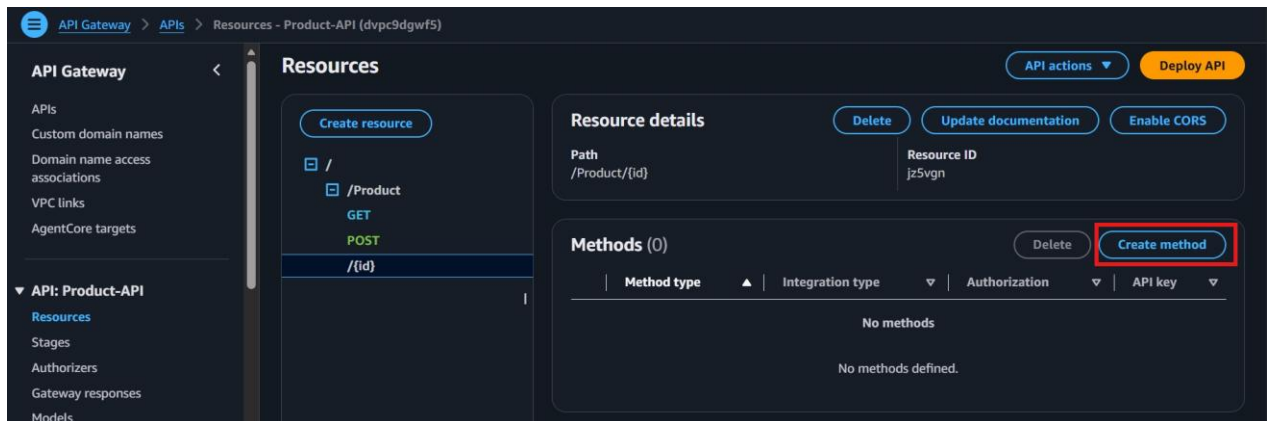
- Click **Create Resource**.



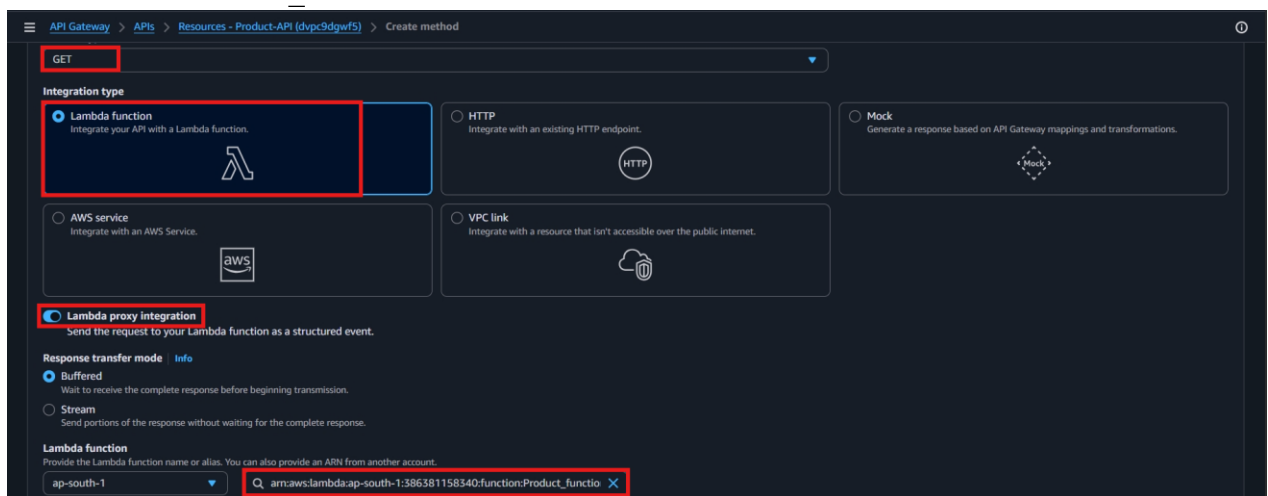
Note: This lab uses standard REST API resources instead of Proxy Resources.

Step 6 – Create GET by ID and DELETE Methods

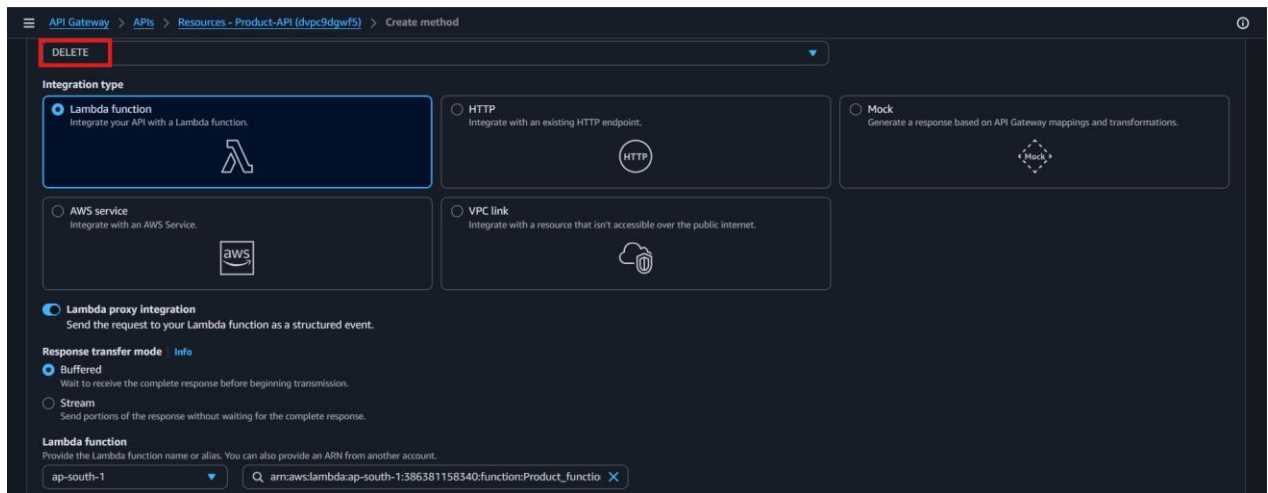
- Under the **/product/{id}** resource:
Create GET Method



- Integration Type: **Lambda Function**
- Enable **Lambda Proxy Integration**
- Select **Product_Function**



- Click **Create method**.
- **Create DELETE Method**
- Integration Type: **Lambda Function**
- Enable **Lambda Proxy Integration**
- Select **Product_Function**

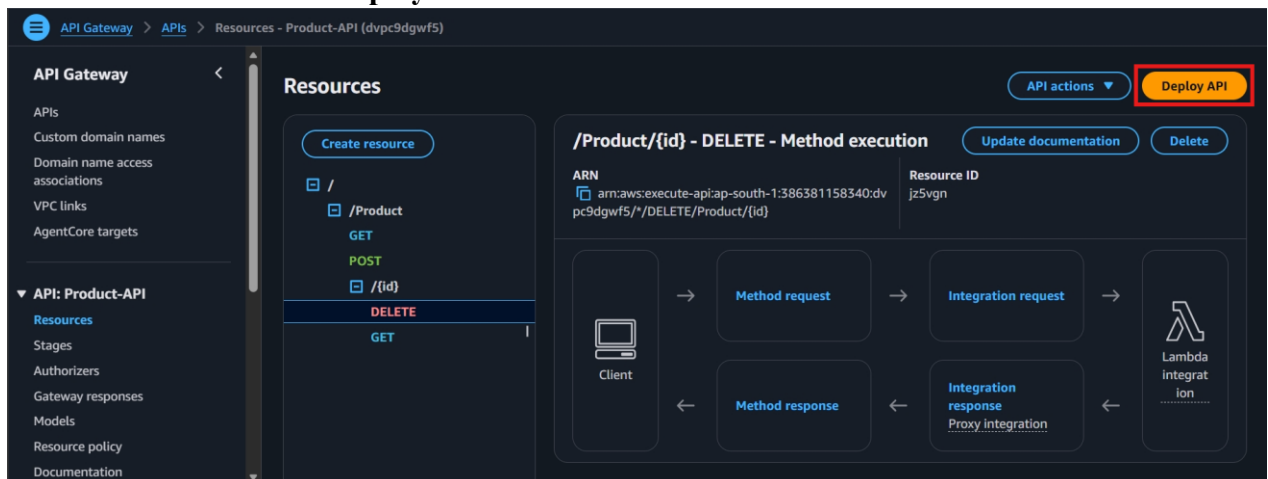


- Click **Create method**.

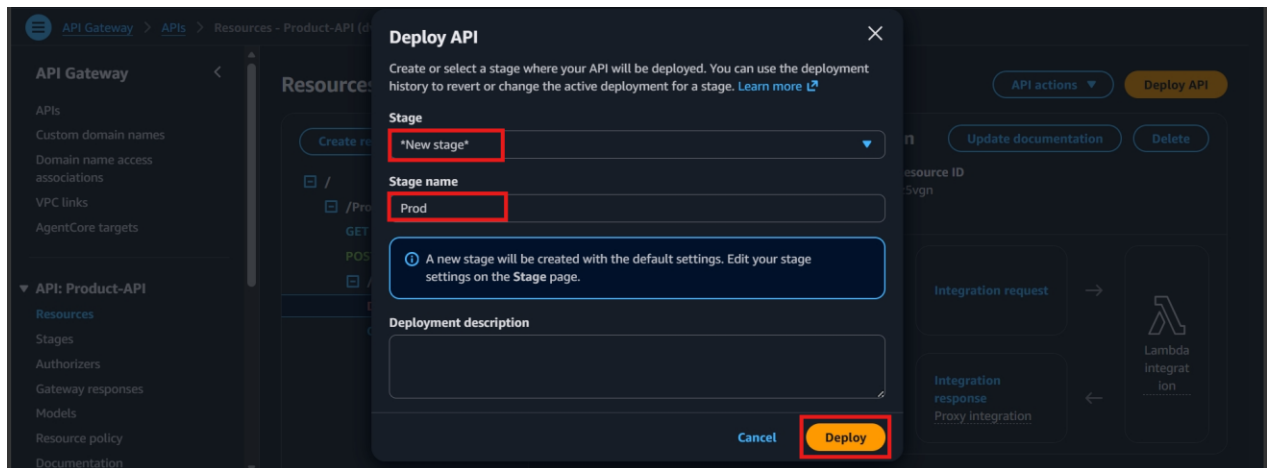
Accept the permission prompt whenever API Gateway requests permission to invoke the Lambda function.

Step 7 – Deploy the REST API

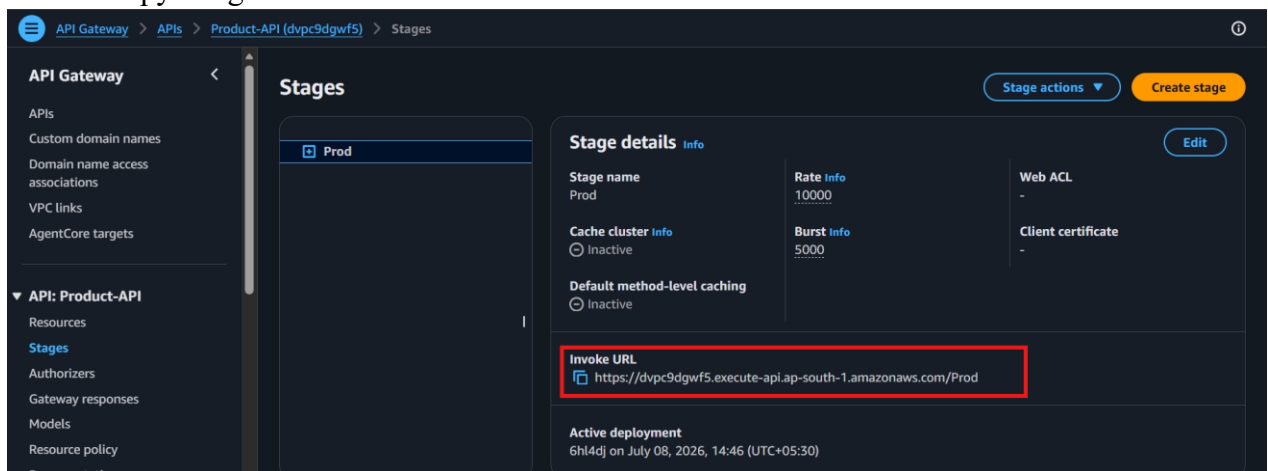
- Select the API.
- Click **Actions** → **Deploy API**.



- Create a new deployment stage.
 - Stage Name: prod



- Click **Deploy**.
- Copy the generated **Invoke URL**.

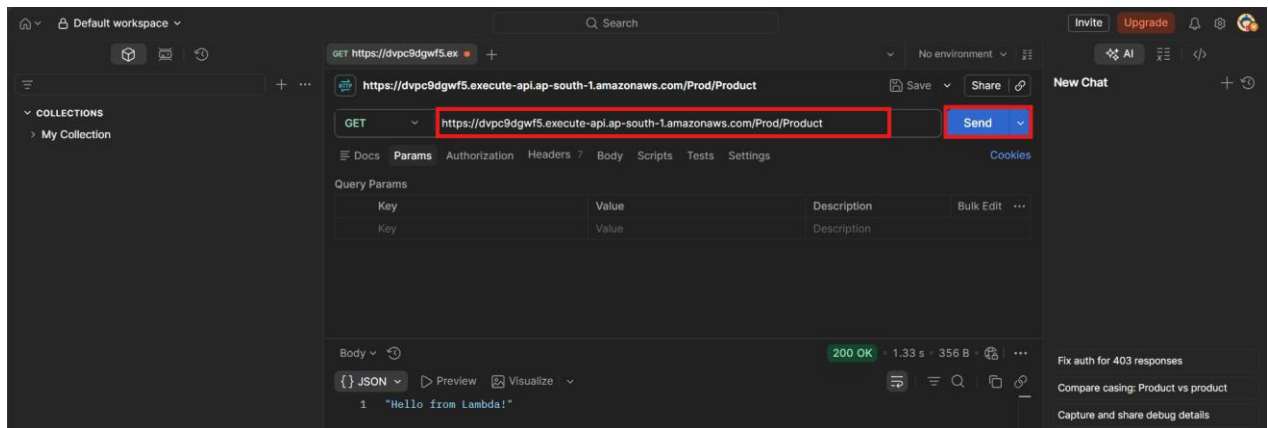


Note: Ensure every HTTP method is integrated with the Lambda function before deploying the API.

Step 8 – Test the REST API

Test GET Request

- Append **/product** to the Invoke URL.
- Open the URL in a web browser or Postman.
- Verify that the Lambda function returns the default response.

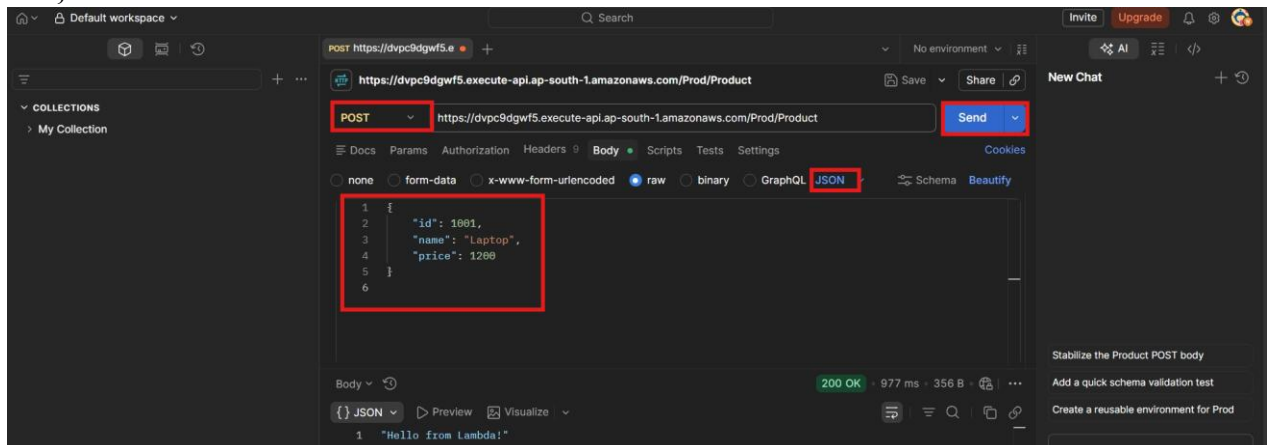


- **Note:** If you open only the Invoke URL, API Gateway returns a Missing Authentication Token error. Append /product to access the configured resource.

Test POST Request

- Open **Postman**.
- Select **POST**.
- Enter the following endpoint:
https://<api-id>.execute-api.<region>.amazonaws.com/prod/product
- Select **Body** → **Raw** → **JSON**.
- Use the following payload:

```
{
  "id": 1001,
  "name": "Laptop",
  "price": 1200
}
```



- Click **Send**.
- Verify that the request is successfully processed.

Step 9 – Capture REST API Event Objects

- Open the **Product_Function** Lambda function.

- Navigate to:
Monitoring → View CloudWatch Logs

The screenshot shows the AWS Lambda console for a function named 'Product_function'. The 'Monitor' tab is active, displaying a 'CloudWatch metrics' section. A red box highlights the 'View CloudWatch logs' button in the top right corner of the monitor section.

- Open the latest log stream.
- Verify that the incoming REST API event object is logged.
- Review the REST API event object and observe fields such as resource, httpMethod, pathParameters, queryStringParameters, requestContext, and body.

The screenshot shows the AWS CloudWatch console for the log stream of 'Product_function'. The 'Log events' section is expanded, showing a log event with the following details:

- Timestamp:** 2026-07-08T09:55:18.722Z
- Message:** 2026-07-08T09:55:18.722Z e3fc06f3-2489-46e8-95d4-86fcacaf2584 INFO {"resource":"/Product","path":"/Product","httpMethod":"POST","headers":{"Accept":"*/","Accept-Encoding":"gzip, deflate, br","Cache-Control":"no-cache","Content-Type":"application/json","Host":"apigateway-south-1.amazonaws.com","Postman-Token":"66079852-470f-4f44-acf0-8c9dc179983","User-Agent":"PostmanRuntime/7.54.0","X-Amzn-Trace-Id":"Root=1-646e1086-08e19885c358623d2807f04","X-Forwarded-For":"49.56.106.113, 54.86.50.139","X-Forwarded-Port":"443","X-Forwarded-Proto":"https"},"queryStringParameters":null,"multiValueQueryStringParameters":null,"pathParameters":null,"stageVariables":null,"requestContext":{"resourceId":"24wlyx","resourcePath":"/Product","httpMethod":"POST","extendedRequestId":"Aml1H9HhucEb10-","requestTime":"08/Jul/2026:09:55:18 +0000","path":"/Prod/Product","accountId":"386381158340","protocol":"HTTP/1.1","stage":"Prod","domainPrefix":"apigateway","requestTimeEpoch":172564530679,"requestId":"9fd12575-58c1-413f-92ce-3cf89a066e60","

The 'httpMethod' field is highlighted with a red box.

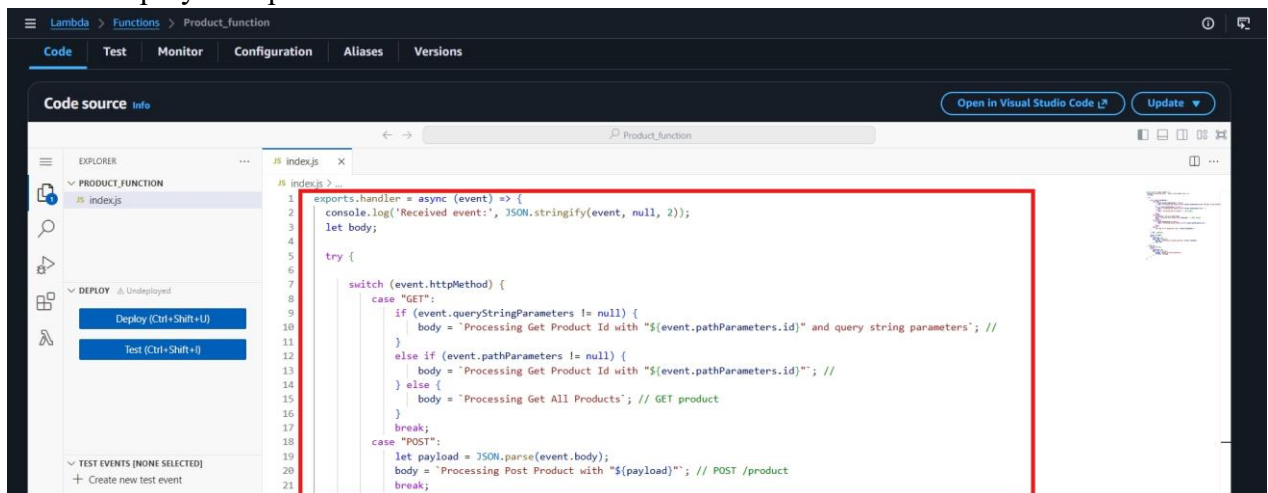
- Repeat the process for:
GET /product
GET /product/{id}

POST /product

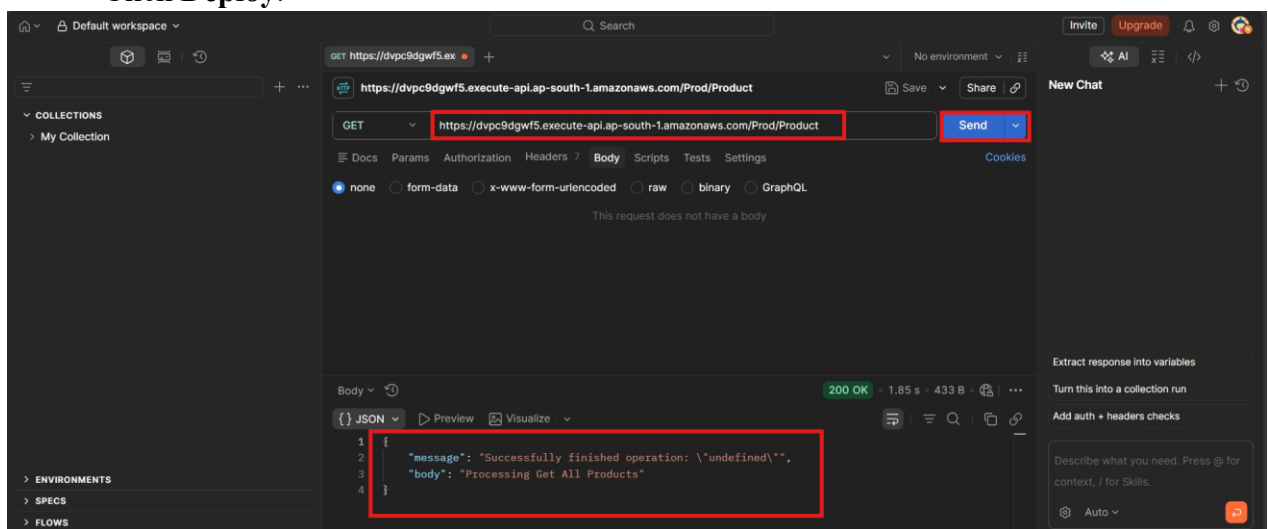
- Compare the event objects generated by each request.

Step 10 – Update the Lambda Function

- Modify the Lambda function to process different HTTP methods.
- Implement logic to handle:
 - GET
 - POST
 - DELETE
- Update the Lambda function to handle GET, POST, and DELETE requests.
- Deploy the updated Lambda function.

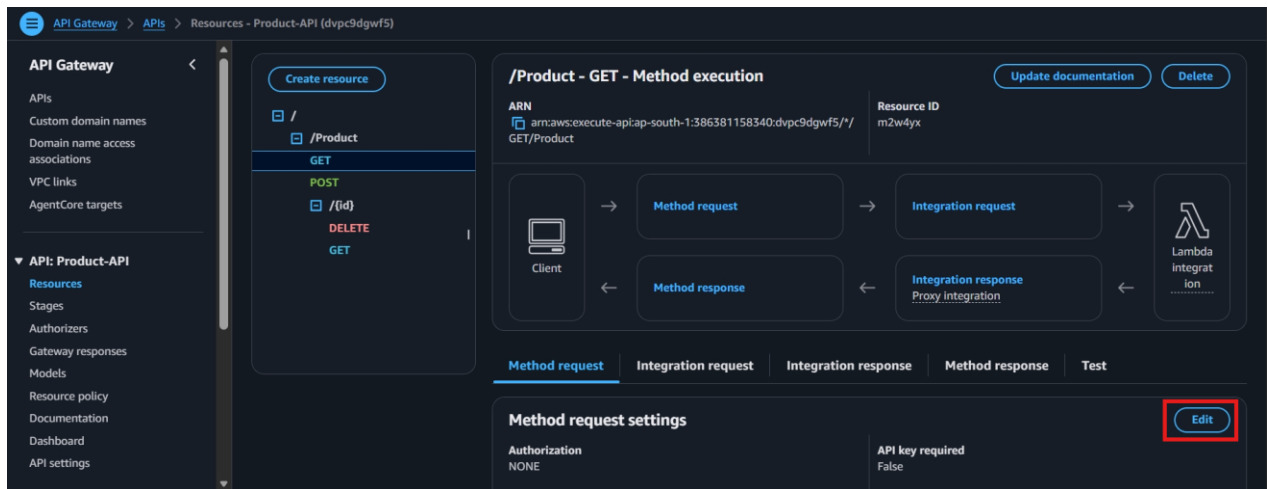


- Click Deploy.

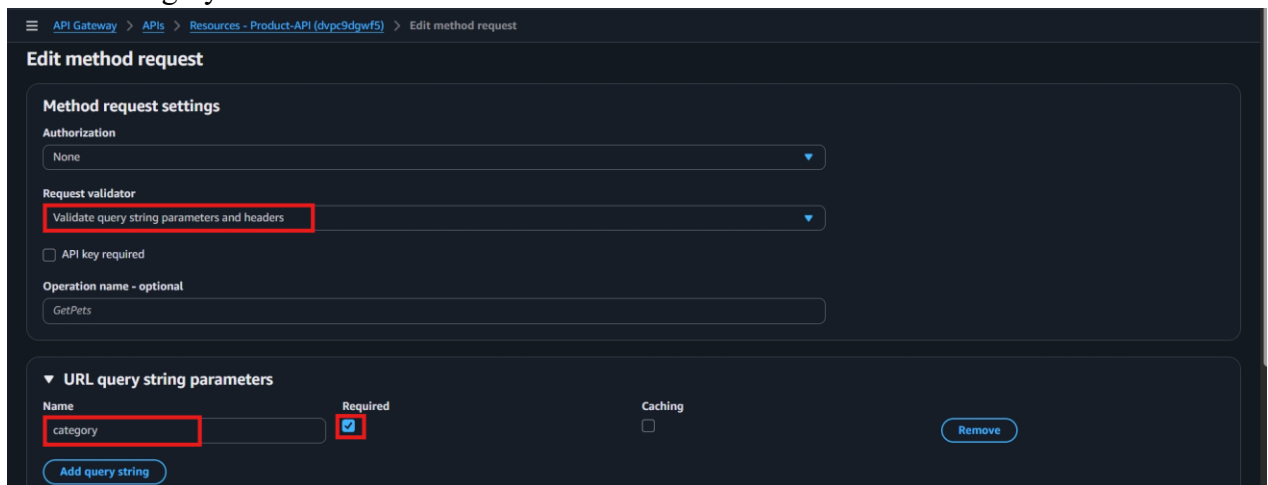


Step 11 – Enable Request Validation

- Open API Gateway.
- Select the GET method under the /product resource.



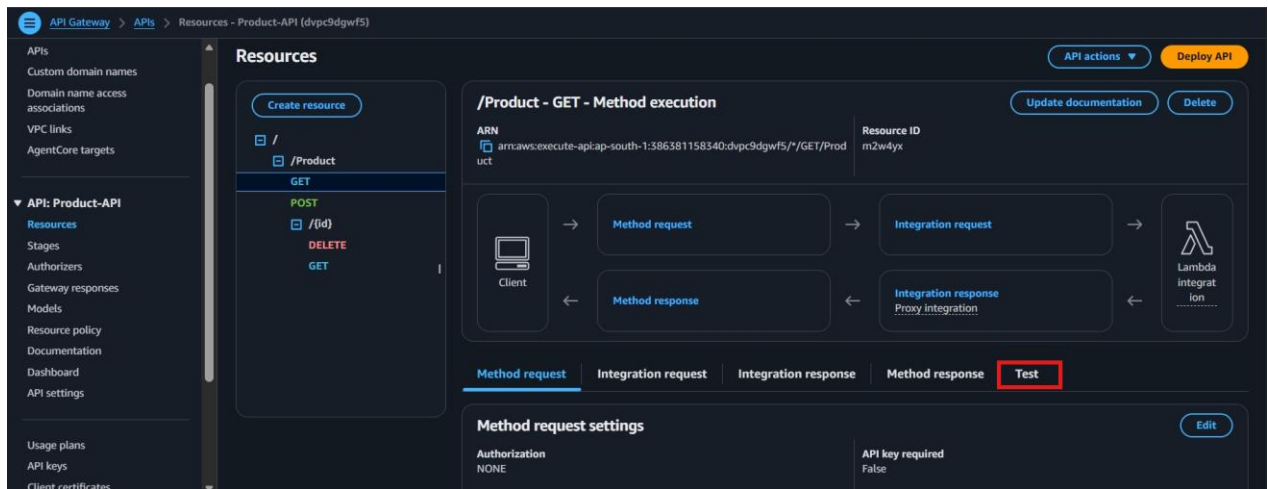
- Click **Method Request**.
- Edit the **Request Validator**.
- Select:
Validate query string parameters and headers
- Expand URL Query String Parameters.
- Add the following required parameter:
Category



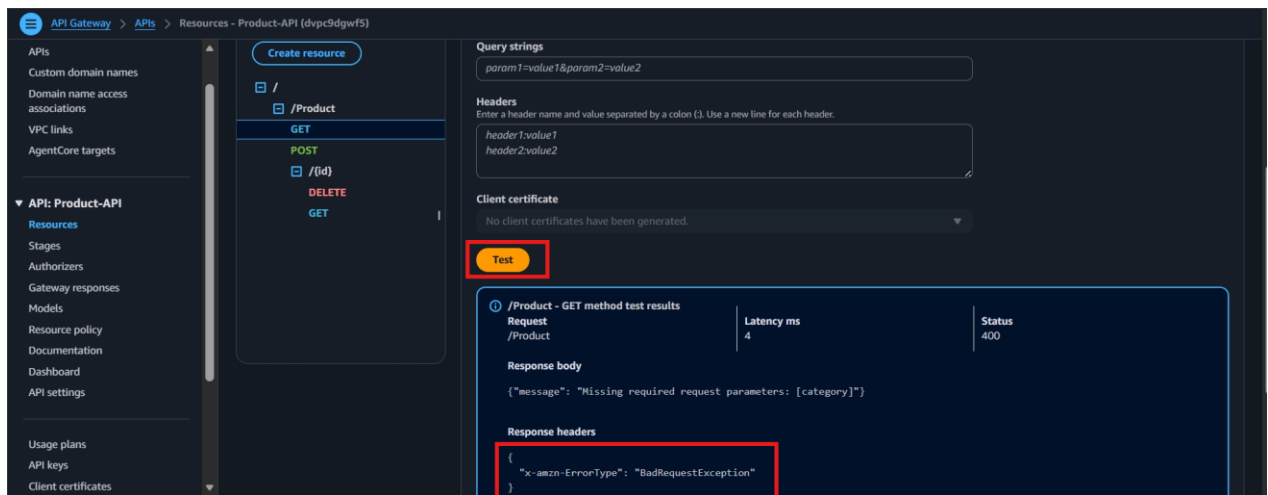
- Save the changes.

Step 12 – Test Request Validation

- From the Method Execution page:
Test without a Query Parameter
- Click **Test**.



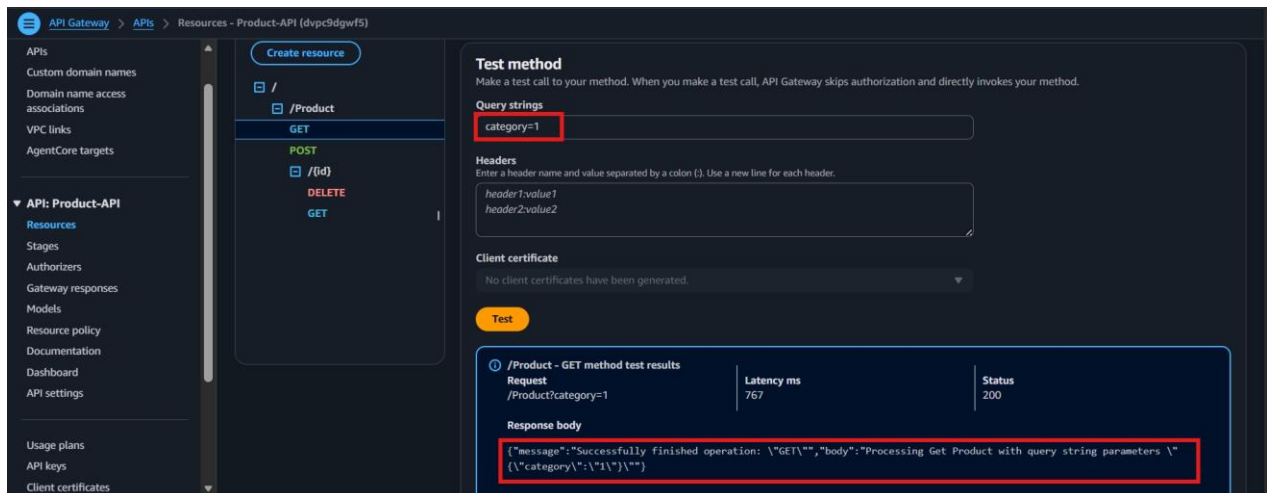
- Leave the query string empty.
- Click Test.



- Verify that the request returns a 400 Bad Request.

Test with a Query Parameter

- Provide:
category=1
Click Test.



- Verify that the request succeeds and returns a **200 OK** response.

Verification

- Lambda function created successfully.
- Amazon API Gateway REST API created successfully.
- Product resources and HTTP methods configured successfully.
- API Gateway integrated with the Lambda function successfully.
- REST API deployed successfully.
- REST API tested successfully using the browser and Postman.
- Incoming REST API event objects captured successfully in CloudWatch Logs.
- Lambda function updated to process GET, POST, and DELETE requests successfully.
- Request validation configured successfully.
- API responses and request validation tested successfully.